

13장

동적 메모리 관리와 표준 함수

- 13.1 동적 메모리 관리
- 13.2 동적 메모리 관리 함수
- 13.3 수학연산 관련 함수
- 13.4 문자열-수치 변환 함수
- 13.5 문자 평가 함수
- 13.6 문자 변환 함수



학습목표

- C 언어에서의 메모리 할당 방식을 설명할 수 있다.
- 변수 종류에 따른 메모리 할당 영역을 설명할 수 있다.
- 메모리의 동적 할당하는 함수를 이용하여 프로그램을 작성할 수 있다.
- 수학연산 함수를 사용할 수 있다.
- 문자열-수치 변환 함수를 사용할 수 있다.
- 문자 평가 함수를 사용할 수 있다.
- 문자 변환 함수를 사용할 수 있다.

동적 메모리 관리

- 메모리의 할당 방식 (1)

- 프로그램은 하드 디스크와 같은 보조기억장치에 저장되어 있다가, 실행을 하면 프로그램의 전체 내용이 메모리로 로드(load)됨
- 메모리로 로드 되는 내용은 프로그램 코드와 코드에서 선언한 데이터 항목(변수, 구조체, 배열 등)이 포함

- 메모리 할당 방식

- 정적 메모리 할당(static memory allocation)

- 메모리 할당 내용들은 프로그램을 작성할 때 미리 정해줌
- 하나의 프로그램에서 사용되는 메모리의 사용량을 프로그램을 실행하기 전(컴파일 할 때)에 미리 필요량을 지정해놓고 할당

- 동적 메모리 할당(dynamic memory allocation)

- 프로그램을 사용하면서 사용자의 요구에 따라 메모리의 필요량이 달라질 때 컴퓨터에 요구해서 메모리를 할당
- 프로그램 실행 시에 메모리를 할당 받으며 그 양은 정해진 한도 내에서 자유롭게 변할 수 있음

동적 메모리 관리

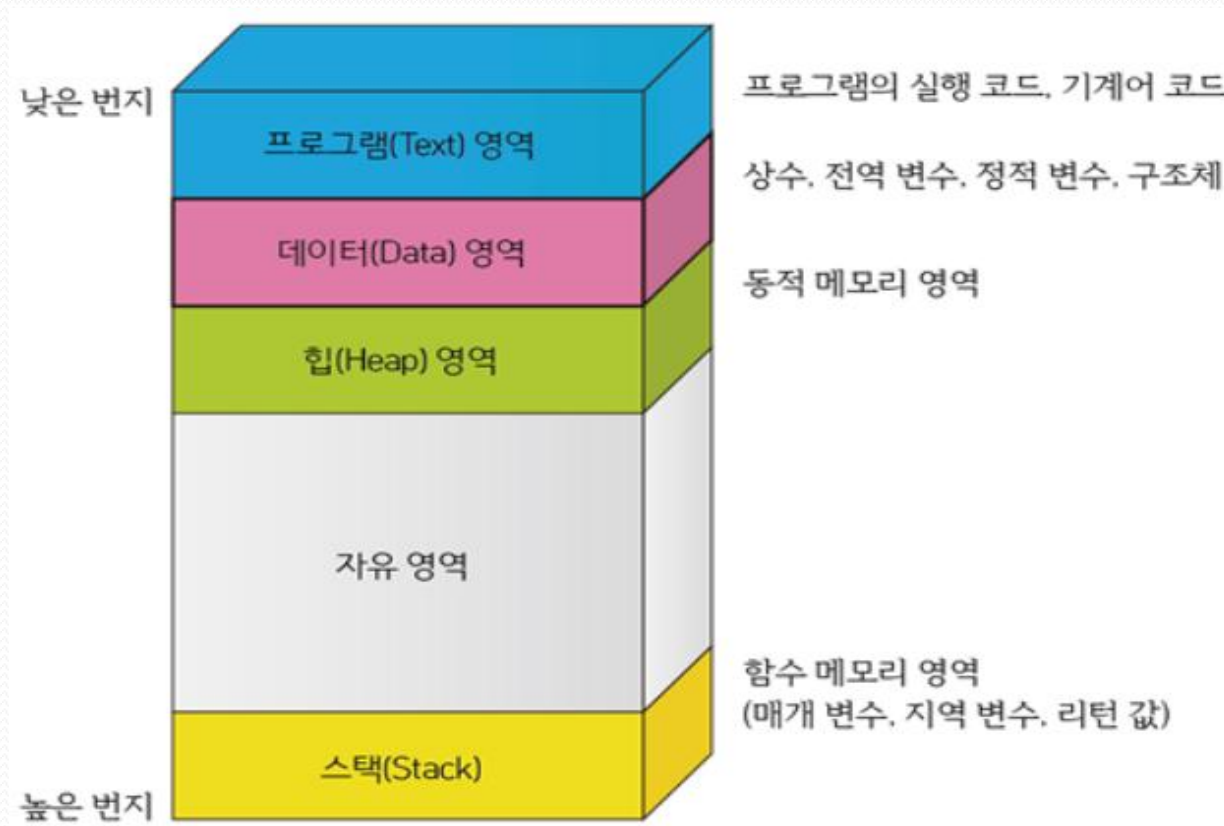
● 메모리의 할당 방식 (2)

- 프로그램의 외부 변수와 정적 변수는 기억장소를 할당 받아 그 값을 저장
- 프로그램 내의 자동변수들은 프로그램 실행 중 필요할 때 기억장소를 할당 받고 그 변수의 통용범위를 넘어서면 기억장소에서 지워짐
- 프로그램의 실행 코드와 모든 변수들은 프로그램이 실행될 때 기억장소에 그 값을 저장
- 메모리 영역에서 변수의 종류에 따른 기억장소의 할당 영역
 - 프로그램(Text) 영역 : 기계어 코드, 프로그램의 실행 코드가 저장되는 영역, 즉, 프로그램 명령이 위치하는 영역
 - 데이터(Data) 영역 : 상수, 전역변수, 구조체가 할당되는 영역. 프로그램 시작과 동시에 메모리에 할당되고 종료시 메모리는 소멸
 - 힙(Heap) 영역 : 동적 메모리 영역, 실행 시 크기 결정
 - 스택(Stack) 영역 : 함수가 호출되는데 필요한 메모리 영역 (매개 변수, 지역 변수, 리턴 값)

동적 메모리 관리

- 메모리의 할당 방식 (3)

- 메모리 영역에서 변수의 종류에 따른 기억장소의 할당 영역



동적 메모리 관리

- 동적 메모리 할당의 필요성

- 동적 메모리 할당이라는 개념은 프로그램이 실행되고 있는 도중에 메모리가 확보됨
 - 즉, 실행 시에 메모리의 크기가 결정됨
- 메모리는 무한한 자원이 아니므로 항상 효율적으로 분배하여 낭비되는 공간이 없게끔 해야 함

예시) 만약 어떤 학교에서 전교생 1000명의 점수를 저장해야 하는 프로그램을 다음과 같이 정수형 배열을 선언해서 점수를 저장한다고 하자.

(예) `int number[1000];`

전교생의 정원이 1200명이 되면 프로그램 소스 코드를 수정

전교생이 500명으로 줄게 되면 500개의 공간은 낭비됨

- 메모리 낭비 문제를 해결하기 위하여 C 언어에서는 프로그램 실행 시 필요할 때마다 새로운 메모리의 공간 할당을 요구하고 필요 없을 시에는 다시 메모리 공간을 반납할 수 있는 동적 할당의 방법을 제공

동적 메모리 관리 함수

- 동적 메모리 관리는 함수로 처리
 - 동적 메모리 관리 함수에는 malloc(), calloc(), realloc(), free() 함수가 있고, 이들은 stdlib.h와 malloc.h 파일에 선언되어 있음.

동적 메모리 관리 함수

```
void *malloc(size_t size);
```

// size 바이트 크기의 기억장소를 할당하고 할당된 주소를 반환한다.

```
void free(void *ptr);
```

// malloc()이나 calloc(), realloc() 함수에 의해 할당된 ptr가 가리키는 기억장소를 해제한다.

```
void *calloc(size_t num, size_t size);
```

// size 바이트 크기의 기억장소를 num 개수만큼 할당하고 그 시작 주소를 반환한다. 0으로 초기화하여 준다.

```
void *realloc(void *pointer, size_t size);
```

// malloc()이나 calloc() 함수에 의하여 할당된 기억장소의 크기를 재조정한다.

동적 메모리 관리 함수

- 동적 메모리 할당 함수 : malloc() (1)

```
#include <stdlib.h>
```

```
void *malloc(size_t size);
```

- 설명 : size 바이트 크기의 기억장소를 할당한다.
- 반환 값 : 정상 실행시 – 할당한 기억장소의 시작 주소를 반환
오류 발생시 – NULL 포인터를 반환

- size 바이트 크기의 기억장소를 할당한다.
- size_t형은 unsigned int형과 동일하며 메모리의 크기를 나타낸다.
- 동적 할당에 성공하면 함수의 반환 자료형은 void * 형이다.
 - void 포인터는 모든 자료형을 가리킬 수 있는 포인터로 주소를 저장하는 용도로만 사용된다.

동적 메모리 관리 함수

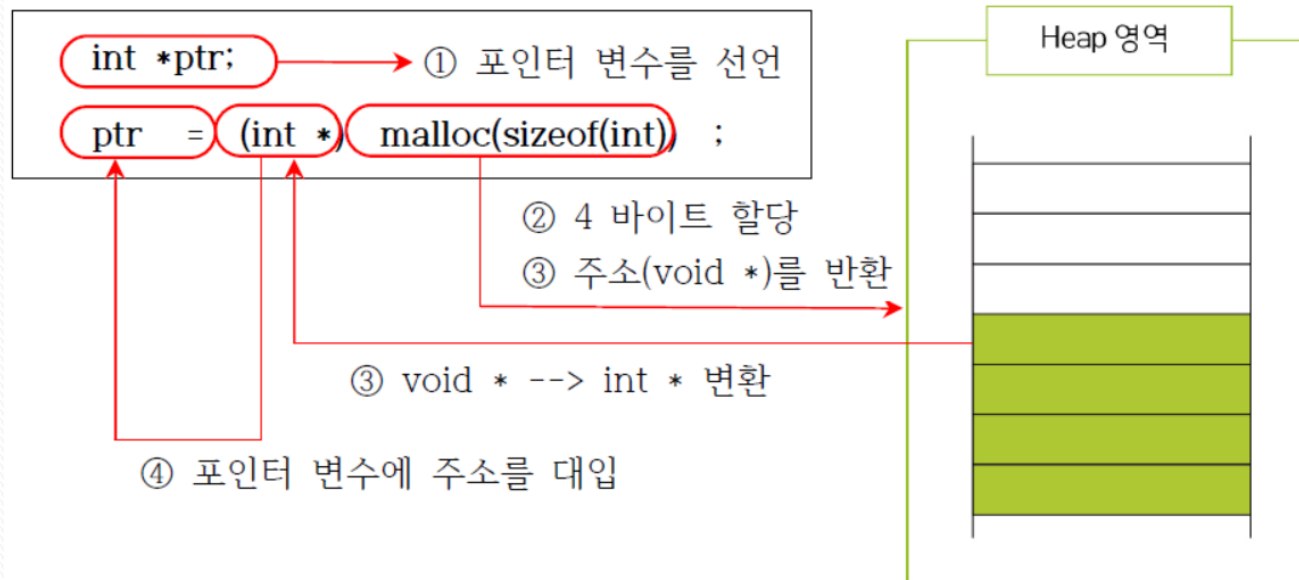
- 동적 메모리 할당 함수 : malloc() (2)

- 할당된 기억공간이 반환되는 void 포인터는 필요한 자료형의 포인터로 cast 연산자를 이용하여 형 변환한 후에 사용

- (예시)

```
int *ptr;
```

```
ptr = (int *)malloc(sizeof(int));
```



동적 메모리 관리 함수

- 동적 메모리 할당 함수 : malloc() (3)

- 할당 받은 기억공간에 데이터를 대입하려면 포인터의 이름 앞에 간접 연산자(*)를 붙여서 사용

- (예) int형 1개를 동적으로 할당 받아 여기에 625라는 정수 값을 대입하는 경우

```
int *nptr = (int *)malloc(sizeof(int));  
*nptr = 625;
```

- (예) char형 15개를 동적으로 할당 받아 여기에 "Hello! World." 이라는 문자열을 복사하는 경우

```
char *sptr = (char *)malloc(15 * sizeof(char));  
strcpy_s(sptr, 15, "Hello! World.");
```

동적 메모리 관리 함수

- 동적 메모리 해제 함수 : free()

- 동적 할당은 프로세스의 힙 영역에 할당하므로 프로세스가 종료되면 운영 체제에 메모리 리소스가 반납되어 해제 됨
- 프로세스가 계속 실행될 때에는 동적 할당된 영역은 유지되므로 언젠가는 메모리 부족 현상이 발생
- 사용이 완료된 영역은 메모리 해제 함수인 free() 함수를 사용하여 메모리를 해제해 주어야 함.

```
#include <stdlib.h>
```

```
void free(void *ptr);
```

- 설명 : malloc()이나 calloc(), realloc() 함수에 의해 할당된 ptr이 가리키는 기억장소를 해제한다.

- (예)

```
free(ptr);
```

동적 메모리 관리 함수

- 예제 13.1 (ex13_1.c) (1)

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    int i, *p;
    p = (int *)malloc(5 * sizeof(int));

    for (i=0; i<5; i++) {
        p[i] = i;
        printf("p[%d]에는 %d이 들어 있습니다. \n", i, p[i]);
    }
    printf("\n");
    for (i=0; i<5; i++) {
        *(p + i) = i;
        printf("*(p+%d)에는 %d이 들어 있습니다. \n", i, *(p + i));
    }
    free(p);
}
```

동적 메모리 관리 함수

- 예제 13.2 (ex13_2.c) (1)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(){
    char *sptr;
    sptr = (char *)malloc(15 * sizeof(char));
    if (sptr == NULL) {
        printf("Insufficient memory \n");
        return 1;
    }

    strcpy_s(sptr, 15, "Hello! World.");      // strcpy(sptr, "Hello! World.");
    printf("%s \n", sptr);
    free(sptr);

    return 0;
}
```

```
Hello! World.
```

```
계속하려면 아무 키나 누르십시오 . . .
```

동적 메모리 관리 함수

- 초기화를 해주는 동적 메모리 할당 함수 : calloc()
 - calloc() 함수는 개체 단위의 배열 형태로 메모리를 동적으로 할당하고 각 원소의 값은 0으로 초기화

```
#include <stdlib.h>
```

```
void *calloc(size_t num, size_t size);
```

- 설명 : 인수 size 크기의 기억장소를 num 개수만큼, 즉 $\text{size} \times \text{num}$ 바이트의 기억장소를 할당하고 그 시작 주소를 반환.

0으로 초기화하여 준다.

- 반환 값 : 정상 수행 시 - 할당한 기억장소의 시작 주소를 반환
오류 발생 시 - NULL 포인터를 반환

- (예)

```
ptr = calloc(10, sizeof(long));
```

```
// long int형의 크기(4 바이트)의 메모리를 10개 동적으로 할당
```

동적 메모리 관리 함수

- 예제 13.3 (ex13_3.c) (1)

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    int i, *p;
    p = (int *)calloc(5, sizeof(int));

    for (i=0; i<5; i++)
        printf("p[%d] : %d ", i, p[i]);
    printf("\n\n");

    for (i=0; i<5; i++) {
        p[i] = i;
        printf("p[%d]에는 %d이 들어 있습니다. \n", i, p[i]);
    }
    printf("\n");

    free(p);
}
```

```
p[0] : 0   p[1] : 0   p[2] : 0   p[3] : 0   p[4] : 0
p[0]에는 0이 들어 있습니다.
p[1]에는 1이 들어 있습니다.
p[2]에는 2이 들어 있습니다.
p[3]에는 3이 들어 있습니다.
p[4]에는 4이 들어 있습니다.
계속하려면 아무 키나 누르십시오 . . .
```

동적 메모리 관리 함수

- 메모리 재할당 함수 : `realloc()` (1)

- `realloc()` 함수는 `malloc()` 이나 `calloc()` 함수를 이용하여 동적으로 메모리를 할당 받았던 메모리의 크기를 재조정할 때 사용

```
#include <stdlib.h>
```

```
void *realloc(void *pointer, size_t size);
```

- 설명 : `malloc()`이나 `calloc()` 함수에 의하여 할당된 기억장소의 크기를 재조정한다. 이미 기억장소를 할당 받은 포인터 `pointer`가 가리키는 기억장소의 크기를 `size` 바이트 크기로 변경하여 그 시작 주소를 반환한다. 새로 할당된 기억장소는 이전과 달라질 수 있으므로 `pointer`는 더 이상 유효하지 않다.
- 반환 값 : 정상 수행 시 - 할당한 기억장소의 시작 주소를 반환
오류 발생 시 - `NULL` 포인터를 반환

동적 메모리 관리 함수

- 메모리 재할당 함수 : `realloc()` (2)

- `realloc()` 함수의 인수 중 `void *pointer`는 원래 동적으로 할당 받았던 메모리를 가리키고 있는 포인터
- 새로 할당할 메모리의 크기는 `size`로 지정
- `realloc()` 함수를 사용해서 메모리를 재할당하면 원래 할당되어 있던 메모리 공간이 아닌 새로운 곳에 새로운 메모리 공간을 확보하게 되는데, 이 때 원래 메모리 공간에 있던 내용들을 새로운 공간에 복사하게 되므로 메모리의 크기가 허용하는 한 데이터의 손실은 발생하지 않음
- 만약 메모리 공간이 부족하다면 `NULL` 포인터 값을 반환
- (예)

```
ptr = (char *)malloc(5 * sizeof(char));  
ptr = (char *)realloc(ptr, 10 * sizeof(char));
```

- 처음에 `malloc()` 함수를 이용하여 문자 5개를 저장할 수 있는 공간을 할당 받음
- 이것을 문자 10개를 저장할 수 있는 공간으로 재할당함

동적 메모리 관리 함수

- 예제 13.4 (ex13_4.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
void main(){
    char *str;

    str = (char *)malloc(5 * sizeof(char));
    strcpy_s(str, 5, "Good");           // strcpy(str, "Good");
    puts(str);

    str = (char *)realloc(str, 15);
    strcat_s(str, 15, " morning");      // strcat(str, " morning");
    puts(str);

    free(str);
```

```
Good
Good morning
계속하려면 아무 키나 누르십시오 . . . _
```

동적 메모리 관리 함수

- 예제 13.5 (ex13_5.c) (1)

```
#include <stdio.h>
#include <stdlib.h>
void main(){
    int i;
    int *pa;
    long long *pb;

    pa = (int *)calloc(5, sizeof(int));
    for (i=0; i<5; i++)
        pa[i] = i;
    for (i=0; i<5; i++)
        printf("%6d ", pa[i]);
    printf("\n");
```

동적 메모리 관리 함수

- 예제 13.5 (ex13_5.c) (2)

```
pb = (long long *)realloc(pa, 5 * sizeof(long long));
for (i=0; i<5; i++)
    pb[i] = i * 1000;
for (i=0; i<5; i++)
    printf("%6lld ", pb[i]);
printf("\n");

free(pb);
}
```

```
    0      1      2      3      4
    0    1000    2000    3000    4000
계속하려면 아무 키나 누르십시오 . . .
```

동적 메모리 관리 함수

- 실습문제 13.1 (1)

- 다음과 같은 동적 메모리 할당의 프로그램을 작성하시오. 크기 5의 int형 배열을 동적으로 생성하고, 배열의 원소에 0, 10, 20, 30, 40을 저장하고 출력한다. 이 배열을 크기 10의 int형 배열로 메모리 재할당하고, 추가로 할당된 원소에는 500, 600, 700, 800, 900을 저장하고 출력한다.

동적 메모리 관리 함수

- 실습문제 13.1 (px13_1.c) (2)

```
#include <stdio.h>
#include <stdlib.h>

void main(){
    int i;
    int *pt;

    pt = (int *)malloc(5 * sizeof(int));
    for (i=0; i<5; i++)
        pt[i] = 10 * i;
    for (i=0; i<5; i++)
        printf("%d ", pt[i]);
    printf("\n");
}
```

동적 메모리 관리 함수

- 실습문제 13.1 (px13_1.c) (3)

```
pt = (int *)realloc(pt, 10 * sizeof(int));
for (i=5; i<10; i++)
    pt[i] = 100 * i;
for (i=0; i<10; i++)
    printf("%d ", pt[i]);
printf("\n");

free(pt);
}
```

```
0 10 20 30 40
0 10 20 30 40 500 600 700 800 900
계속하려면 아무 키나 누르십시오 . . .
```

동적 메모리 관리 함수

- 실습문제 13.2 (1)

- 다음과 같은 동적 메모리 할당의 프로그램을 작성하시오. 크기 5의 int형 배열을 calloc() 함수를 사용하여 동적으로 생성하고, 다시 이 배열을 크기 10의 int형 배열로 메모리 재할당하고, 추가로 할당된 원소에는 500, 600, 700, 800, 900을 저장하고 출력하는 프로그램으로 고쳐 작성하시오.

동적 메모리 관리 함수

- 실습문제 13.2 (px13_2.c) (2)

```
#include <stdio.h>
#include <stdlib.h>

void main() {
    int i;
    int *pt;

    pt = (int *)calloc(5, sizeof(int));
    pt = (int *)realloc(pt, 10 * sizeof(int));
    for (i=5; i<10; i++)
        pt[i] = 100 * i;
    for (i=0; i<10; i++)
        printf("%d ", pt[i]);
    printf("\n");

    free(pt);
}
```

```
0 0 0 0 0 500 600 700 800 900
계속하려면 아무 키나 누르십시오 . . .
```

수학연산 관련 함수

- 수학연산 관련 함수 (1)

- 삼각함수, 제곱근, 지수, 대수, 절대값 등의 수학 관련 함수는 헤더 파일 `<math.h>`에 선언되어 있음

수학연산 관련 함수

```
double sin(double x);
```

```
// sin 값을 구한다. x는 라디안으로 지정되는 각도이다.
```

```
double cos(double x);
```

```
// cos 값을 구한다. x는 라디안으로 지정되는 각도이다.
```

```
double tan(double x);
```

```
// tan 값을 구한다. x는 라디안으로 지정되는 각도이다.
```

수학연산 관련 함수

● 수학연산 관련 함수 (2)

수학연산 관련 함수

double exp(double x);
// x의 자연 지수 e^x 값을 구한다.

double log(double x);
// x의 자연 대수 $\log_e x$ 값을 구한다.

double log10(double x);
// x의 상용 대수 $\log_{10} x$ 값을 구한다.

double sqrt(double x);
// x의 제곱근을 구한다.

double pow(double x, double y);
// x의 y 승(x^y)을 구한다.

int abs(int x);
// 정수 x의 절대값을 구한다.

수학연산 관련 함수

- 수학연산 관련 함수 (3)

수학연산 관련 함수

```
double ceil(double x);  
    // 실수 x의 소수 이하를 올린 값을 구한다.
```

```
double floor(double x);  
    // 실수 x의 소수 이하를 버린 값을 구한다.
```

```
double fmod(double x, double y);  
    // 실수 x를 y로 나누었을 때(x/y)의 나머지를 구한다.
```

```
double modf(double x, double *intptr);  
    // 실수 x를 소수부(반환 값)와 정수부(*intptr)로 분할한다.
```

수학연산 관련 함수

- 예제 13.6 (ex13_6.c) (1)

```
#include <stdio.h>
#include <math.h>
const double pi = 3.14159;
void main() {
    int deg1 = 30, deg2 = 45;
    double rad1, rad2;

    rad1 = deg1 * pi / 180;
    rad2 = deg2 * pi / 180;

    printf("sin(%d) = %lf \n", deg1, sin(rad1));
    printf("cos(%d) = %lf \n", deg1, cos(rad1));
    printf("tan(%d) = %lf \n", deg1, tan(rad1));
    printf("\n");
}
```

수학연산 관련 함수

- 예제 13.6 (ex13_6.c) (2)

```
printf("sin(%d) = %lf \n", deg2, sin(rad2));  
printf("cos(%d) = %lf \n", deg2, cos(rad2));  
printf("tan(%d) = %lf \n", deg2, tan(rad2));  
}
```

```
sin(30) = 0.500000  
cos(30) = 0.866026  
tan(30) = 0.577350  
  
sin(45) = 0.707106  
cos(45) = 0.707107  
tan(45) = 0.999999  
계속하려면 아무 키나 누르십시오 . . .
```

수학연산 관련 함수

- 예제 13.7 (ex13_7.c)

```
#include <stdio.h>
#include <math.h>
void main(){
    double a = 2, b = 25;

    printf("pow(5, 3) = %.1lf \n", pow(5.0, 3.0));
    printf("sqrt(25) = %.1lf \n", sqrt(25.0));
    printf("log10(1000) = %.1lf \n", log10(1000.0));
}
```

```
pow(5, 3) = 125.0
sqrt(25) = 5.0
log10(1000) = 3.0
계속하려면 아무 키나 누르십시오 . . .
```

수학연산 관련 함수

- 예제 13.8 (ex13_8.c) (1)

```
#include <stdio.h>
#include <math.h>
void main() {
    double a, b, c;
    double x, y;

    a = -1.0;
    x = exp(a);
    printf("exp(%lf) = %lf\n", a, x);

    b = log(x);
    printf("log(%lf) = %lf\n\n", x, b);

    a = 1.0;
    y = exp(a);
    printf("exp(%lf) = %lf\n", a, y);
```


수학연산 관련 함수

- 예제 13.8 (ex13_8.c) (2)

```
c = log(y);  
printf("log(%lf) = %lf\n", y, c);  
}
```

```
exp(-1.000000) = 0.367879  
log(0.367879) = -1.000000
```

```
exp(1.000000) = 2.718282  
log(2.718282) = 1.000000
```

```
계속하려면 아무 키나 누르십시오 . . . _
```

수학연산 관련 함수

- 예제 13.9 (ex13_9.c)

```
#include <stdio.h>
#include <math.h>
void main()
{
    double x1 = 65.156, x2 = 65.876;
    double y, z;

    printf("ceil(%lf) = %lf \Wn", x1, ceil(x1));
    printf("ceil(%lf) = %lf \Wn\Wn", x2, ceil(x2));

    printf("floor(%lf) = %lf \Wn", x1, floor(x1));
    printf("floor(%lf) = %lf \Wn\Wn", x2, floor(x2));

    printf("fmod(%lf, %lf) = %lf \Wn", x1, y = 2.4, fmod(x1, y));
    z = modf(x1, &y);
    printf("z = modf(%lf, &y) --> %lf(z) + %lf(y) \Wn", x1, z, y);
}
```

```
ceil(65.156000) = 66.000000
ceil(65.876000) = 66.000000

floor(65.156000) = 65.000000
floor(65.876000) = 65.000000

fmod(65.156000, 2.400000) = 0.356000
z = modf(65.156000, &y) --> 0.156000(z) + 65.000000(y)
계속하려면 아무 키나 누르십시오 . . .
```

문자열-수치 변환 함수

- 문자열-수치 변환 함수

- 문자열을 수치로 변환하는 함수들은 헤더 파일 <stdlib.h>에 선언되어 있음
- 수치 데이터를 문자열로 변환은 sprintf_s() 함수를 사용

문자열-수치 데이터 변환 함수

```
int atoi(const char *str);  
    // str이 가리키는 문자열을 int형 정수로 변환한다.  
    // 변환할 수 없는 문자열을 만나면 변환을 중단하고 0을 반환.
```

```
double atof(const char *str);  
    // str이 가리키는 문자열을 double형 부동소수점 수로 변환
```

```
long atol(const char *str);  
    // str이 가리키는 문자열을 long형 정수로 변환한다.  
    // 변환할 수 없는 문자열을 만나면 변환을 중단하고 0을 반환
```

문자열-수치 변환 함수

- 예제 13.10 (ex13_10.c)

```
#include <stdio.h>
#include <stdlib.h>
void main(){
    int n1;
    long int n2;
    double f1;
    char *str1 = "1234.56";

    n1 = atoi(str1);
    printf("string = %s, integer = %d\n", str1, n1);

    n2 = atol(str1);
    printf("string = %s, long integer = %ld\n", str1, n2);

    f1 = atof(str1);
    printf("string = %s, double = %lf\n", str1, f1);
}
```

```
string = 1234.56, integer = 1234
string = 1234.56, long integer = 1234
string = 1234.56, double = 1234.560000
계속하려면 아무 키나 누르십시오 . . .
```

문자 평가 함수

● 문자 평가 함수 (1)

- 한 개의 문자가 숫자인지, 문자인지, 대문자인지, 소문자인지를 검사하는 함수들은 C 언어에서는 <ctype.h> 파일에 선언되어 있음
- 평가한 항목이 만족하면 참(true)으로 0이 아닌 값을, 만족하지 않으면 거짓(false)으로 0을 반환

문자 평가 함수

```
int isalnum(int c);  
    // 문자 c가 알파벳이나 숫자(A~Z, a~z, 0~9)이면 참
```

```
int isalpha(int c);  
    // 문자 c가 알파벳(A~Z, a~z)이면 참
```

```
int isdigit(int c);  
    // 문자 c가 10진 숫자(0~9)이면 참
```

```
int isxdigit(int c);  
    // 문자 c가 16진 숫자(0~9, A~F, a~f)이면 참
```

문자 평가 함수

- 문자 평가 함수 (2)

문자 평가 함수

```
int isspace(int c);  
    // 문자 c가 공백 문자(공백, 탭)이면 참
```

```
int isupper(int c);  
    // 문자 c가 대문자(A~Z)이면 참
```

```
int islower(int c);  
    // 문자 c가 소문자(a~z)이면 참
```

```
int isprint(int c);  
    // 문자 c가 인쇄 가능 문자이면 참
```

문자 평가 함수

- 예제 13.11 (ex13_11.c) (1)

```
#include <stdio.h>
#include <ctype.h>
void main() {
    char c;

    c = 'A';
    if (isalpha(c)) {
        if (isupper(c))
            printf("%c is alphabetic and upper case.\n", c);
        else
            printf("%c is alphabetic and lower case.\n", c);
    }
    else
        printf("%c is not alphabetic.\n", c);
}
```

문자 평가 함수

- 예제 13.11 (ex13_11.c) (2)

```
c = '5';  
if (isdigit(c))  
    printf("%c is digit.\n", c);  
else  
    printf("%c is not digit.\n", c);  
}
```

```
A is alphabetic and upper case.  
5 is digit.  
계속하려면 아무 키나 누르십시오 . . .
```


문자 변환 함수

● 문자 변환 함수

- 문자 변환 함수는 대문자를 소문자로 변환하거나 소문자를 대문자로 변환. 또는 ASCII 코드가 아닌 값을 ASCII 코드로 변환
- 문자 변환 함수는 <ctype.h> 파일에 선언되어 있음

문자 변환 함수

```
int tolower(int c);  
    // 문자 c가 대문자이면 소문자로 변환한다.  
    // 대문자가 아니면 변환하지 않는다.
```

```
int toupper(int c);  
    // 문자 c가 소문자이면 대문자로 변환한다.  
    // 소문자가 아니면 변환하지 않는다.
```

```
int toascii(int c);  
    // 문자를 아스키 형식으로 변환한다.  
    // 문자 c에서 하위 7비트만 취하여 ASCII 형식  
    // (0~127 사이의 값)으로 변환한다.  
    // 나머지 비트는 0으로 변경한다.
```

문자 변환 함수

- 예제 13.12 (ex13_12.c) (1)

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
void main() {
```

```
    int ascii, cap, low;
```

```
    char ch;
```

```
    printf("문자를 입력하세요 : ");
```

```
    scanf_s("%c", &ch, 1);
```

```
// scanf("%c", &ch);
```

```
    ascii = toascii(ch);
```

```
    printf("ascii = %d \n", ascii);
```

```
    cap = toupper(ch);
```

```
    printf("capital = %c \n", cap);
```

```
    low = tolower(ch);
```

```
    printf("lower = %c \n", low);
```

```
}
```

문자 변환 함수

- 예제 13.12 (ex13_12.c) (2)

```
문자를 입력하세요 : a
ascii = 97
capital = A
lower = a
계속하려면 아무 키나 누르십시오 . . .
```

```
문자를 입력하세요 : A
ascii = 65
capital = A
lower = a
계속하려면 아무 키나 누르십시오 . . .
```

```
문자를 입력하세요 : 7
ascii = 55
capital = 7
lower = 7
계속하려면 아무 키나 누르십시오 . . .
```

동적 메모리 관리 함수

- 실습문제 13.3 (1)

- 한 변의 길이가 20인 정사각형의 대각선의 길이를 구하고, 이 대각선을 한 변의 길이로 하는 정사각형의 넓이를 구하는 프로그램을 작성하시오. 길이와 넓이는 함수를 사용하여 구하시오.

동적 메모리 관리 함수

- 실습문제 13.3 (px13_3.c) (2)

```
#include <stdio.h>
#include <math.h>
void main(){
    double a = 20.0;
    double diag, area;

    diag = sqrt(a * a + a * a);
    area = pow(diag, 2);

    printf("대각선의 길이 : %lf \\\n", diag);
    printf("넓이 : %lf \\\n", area);
}
```

```
대각선의 길이 : 28.284271
넓이 : 800.000000
계속하려면 아무 키나 누르십시오 . . .
```

Question & Answer

